

Programación Avanzada

Trabajo Práctico N.º 2

Proyecto grupal integrador de Programación Orientada a Objetos

Consigna para entrega grupal

Aplicación práctica de clases, objetos, encapsulamiento, herencia, polimorfismo y relaciones entre clases.

Modalidad	Grupal
Entrega	Repositorio de GitHub
Producto esperado	Proyecto funcional aplicando POO
Formato sugerido	Consola, API REST simple o aplicación web sencilla

Materia: Programación Avanzada | UNAB

1. Consigna general

El presente trabajo práctico tiene como objetivo que los estudiantes desarrollen, en grupos, un proyecto funcional aplicando los conceptos principales de Programación Orientada a Objetos vistos en clase.

Cada grupo deberá diseñar, implementar y presentar un sistema desarrollado principalmente bajo el paradigma de POO. El proyecto podrá ser una aplicación de consola bien estructurada, una API REST sencilla, o una aplicación web simple, siempre que se evidencie el uso correcto de clases, objetos, métodos, atributos y relaciones entre entidades.

La entrega deberá realizarse mediante un repositorio en GitHub, donde se pueda verificar la participación de todos los integrantes del equipo a través de commits individuales.

Idea central

No se busca un sistema enorme ni complejo. Se busca un proyecto ordenado, entendible, funcional y correctamente modelado. Mejor algo chico y bien hecho que algo gigante atado con alambre.

2. Modalidad de trabajo

El trabajo será grupal. Cada grupo deberá organizarse internamente para distribuir las tareas de análisis, diseño, programación, prueba y documentación del proyecto.

Todos los integrantes deberán participar activamente en el repositorio de GitHub. No alcanza con figurar en la portada: cada miembro debe tener al menos una contribución visible mediante commits.

3. Objetivo del trabajo

El objetivo principal es construir un proyecto sencillo pero bien armado, donde se apliquen correctamente los conceptos de Programación Orientada a Objetos.

El proyecto deberá demostrar, como mínimo, el uso de:

- Clases.
- Objetos.
- Atributos.
- Métodos.
- Constructores.
- Encapsulamiento.
- Herencia, cuando corresponda.
- Polimorfismo, cuando corresponda.
- Composición o agregación entre clases.
- Separación de responsabilidades.
- Organización del código en archivos o módulos.
- Buenas prácticas básicas de programación.

4. Tipo de proyecto permitido

Opción 1: Proyecto de consola

Consiste en desarrollar una aplicación que funcione desde la terminal o consola. Debe permitir al usuario interactuar mediante un menú de opciones, cargar datos, consultar información, modificar registros y ejecutar operaciones propias del sistema elegido.

Esta opción es la recomendada para quienes quieran concentrarse especialmente en la lógica de Programación Orientada a Objetos.

Ejemplos posibles:

- Sistema de gestión de biblioteca.
- Sistema de gestión de alumnos y materias.
- Sistema de turnos médicos.
- Sistema de ventas simple.
- Sistema de alquiler de vehículos.
- Sistema de stock de productos.
- Sistema de reservas para un hotel.
- Sistema de administración de mascotas en una veterinaria.
- Sistema de gestión de clubes deportivos.
- Sistema de control de gastos personales.

Opción 2: API REST sencilla

Los grupos que quieran investigar un poco más podrán desarrollar una API REST simple, por ejemplo, un ABM. La API puede permitir crear, listar, modificar y eliminar registros de una entidad principal.

Esta opción puede desarrollarse en Python utilizando POO, por ejemplo con Flask o FastAPI, siempre que la lógica principal esté organizada en clases.

- ABM de productos.
- ABM de clientes.
- ABM de alumnos.
- ABM de turnos.
- ABM de libros.
- ABM de vehículos.
- ABM de usuarios.
- ABM de tareas.

Opción 3: Aplicación web simple

También se permite desarrollar una aplicación web sencilla, con una interfaz básica para interactuar con el sistema. No es necesario realizar un diseño avanzado. Lo importante es que el proyecto tenga una estructura clara y que el código aplique conceptos de POO.

- Web para cargar productos.
- Web para administrar alumnos.
- Web para registrar turnos.
- Web para gestionar tareas.
- Web para controlar stock.
- Web para registrar gastos.
- Web para administrar reservas.

5. Requisitos mínimos obligatorios

1. Debe estar desarrollado en grupo.
2. Debe estar subido a un repositorio de GitHub.
3. El repositorio debe incluir a todos los integrantes como colaboradores o evidenciar sus aportes mediante commits.

4. Debe incluir un archivo README.md con nombre del proyecto, descripción, integrantes, tecnologías, instrucciones de ejecución, funcionalidades y explicación breve de las clases utilizadas.
5. El sistema debe tener al menos tres clases propias.
6. Debe existir relación entre clases.
7. Debe incluir un menú, endpoints o interfaz que permita probar las funcionalidades principales.
8. El código debe estar ordenado en carpetas o archivos separados.
9. Debe poder ejecutarse correctamente.
10. Debe evidenciar conceptos de Programación Orientada a Objetos vistos en clase.

6. Conceptos de POO que deben aplicarse

Clase: Debe existir una definición clara de clases que representen entidades del sistema. Ejemplo: Producto, Cliente, Factura, Alumno, Turno, Vehículo.

Objeto: El sistema debe crear objetos a partir de las clases definidas. Ejemplo: crear un nuevo producto, registrar un cliente o cargar un vehículo.

Atributos: Cada clase debe tener atributos que representen sus características. Ejemplo: nombre, precio, stock, categoría.

Métodos: Cada clase debe incluir métodos que representen comportamientos. Ejemplo: actualizar_stock(), calcular_total(), mostrar_datos().

Encapsulamiento: Se debe intentar proteger o controlar el acceso a los datos internos de los objetos, utilizando métodos para consultar o modificar información cuando sea necesario.

Herencia: Cuando el proyecto lo permita, se recomienda utilizar herencia. Ejemplo: Vehículo como clase base y Auto, Moto o Camioneta como clases derivadas.

Polimorfismo: Cuando sea posible, se recomienda aplicar polimorfismo mediante métodos con el mismo nombre pero distinto comportamiento según el tipo de objeto.

Composición o agregación: Debe existir alguna relación entre clases. Ejemplo: una Factura tiene varios Producto, un Cliente tiene varios Turno, o un Pedido contiene varios ItemPedido.

7. Sugerencias de proyectos sencillos

Sistema de biblioteca

Clases posibles: Libro, Usuario, Prestamo, Biblioteca.

Funcionalidades sugeridas:

- Registrar libros.
- Registrar usuarios.
- Prestar libros.
- Devolver libros.
- Listar libros disponibles.
- Consultar préstamos activos.

Sistema de turnos médicos

Clases posibles: Paciente, Medico, Turno, Especialidad.

Funcionalidades sugeridas:

- Registrar pacientes.
- Registrar médicos.
- Crear turnos.
- Cancelar turnos.
- Listar turnos por médico.
- Listar turnos por paciente.

Sistema de ventas

Clases posibles: Producto, Cliente, Venta, DetalleVenta.

Funcionalidades sugeridas:

- Cargar productos.
- Registrar clientes.
- Crear una venta.
- Calcular total.
- Aplicar descuentos.
- Actualizar stock.

Sistema de alquiler de vehículos

Clases posibles: Vehiculo, Auto, Moto, Cliente, Alquiler.

Funcionalidades sugeridas:

- Registrar vehículos.
- Registrar clientes.
- Alquilar vehículo.
- Devolver vehículo.
- Calcular costo del alquiler.
- Listar vehículos disponibles.

Sistema de gestión de alumnos

Clases posibles: Alumno, Materia, Inscripcion, Profesor.

Funcionalidades sugeridas:

- Registrar alumnos.
- Registrar materias.
- Inscribir alumnos.
- Cargar notas.
- Calcular promedio.
- Mostrar materias aprobadas.

Sistema de tareas

Clases posibles: Usuario, Tarea, Proyecto, Categoria.

Funcionalidades sugeridas:

- Crear tareas.
- Asignar tareas a usuarios.
- Cambiar estado de una tarea.
- Listar tareas pendientes.
- Listar tareas finalizadas.

8. Requisitos del repositorio GitHub

El repositorio deberá estar organizado de manera clara. La estructura puede variar según el proyecto, pero debe ser entendible.

```
trabajo-practico-2/  
|  
|-- README.md  
|-- main.py  
|-- models/  
|   |-- producto.py  
|   |-- cliente.py  
|   |-- venta.py  
|  
|-- services/  
|   |-- venta_service.py  
|  
|-- docs/  
|   |-- diagrama_clases.png
```

El archivo README.md es obligatorio y debe permitir que cualquier docente o compañero pueda entender cómo ejecutar y probar el proyecto.

9. Participación en GitHub

Cada integrante deberá realizar commits propios en el repositorio. Se recomienda que los commits sean claros y descriptivos.

Ejemplos de commits correctos:

```
git commit -m "Agrega clase Producto"  
git commit -m "Implementa menú principal"  
git commit -m "Agrega validaciones para ventas"  
git commit -m "Crea README con instrucciones del proyecto"  
git commit -m "Agrega relación entre Cliente y Venta"
```

Ejemplos de commits poco recomendables:

```
git commit -m "cosas"  
git commit -m "final"  
git commit -m "arreglos"  
git commit -m "asdf"
```

10. Entrega

La entrega deberá incluir:

11. Link al repositorio de GitHub.
12. Nombre completo de los integrantes.
13. Breve descripción del proyecto.
14. Instrucciones para ejecutar el sistema.
15. Capturas de pantalla o ejemplos de uso, si corresponde.
16. Explicación breve de las clases principales.
17. Diagrama de clases simple, realizado en UML o similar.

11. Diagrama de clases

Cada grupo deberá incluir un diagrama de clases básico que represente la estructura principal del proyecto.

El diagrama puede realizarse con herramientas como Draw.io, Lucidchart, StarUML, PlantUML, Canva o Diagrams.net. También puede hacerse a mano y digitalizarse, siempre que sea claro.

El diagrama debe mostrar clases principales, atributos, métodos y relaciones entre clases. No es necesario que sea perfecto, pero sí debe reflejar correctamente el diseño del sistema.

12. Criterios de evaluación

Criterio	Descripción
Aplicación de POO	Uso correcto de clases, objetos, métodos y relaciones.
Organización del código	Separación en archivos, claridad y estructura.
Funcionamiento	El sistema debe poder ejecutarse y probarse.
GitHub	Repositorio ordenado y con participación de los integrantes.
README	Documentación clara del proyecto.
Diagrama de clases	Representación básica del diseño del sistema.
Creatividad	Propuesta propia y resolución del problema elegido.
Presentación	Claridad al explicar el proyecto y las decisiones tomadas.

13. Recomendación final

Se recomienda comenzar con una idea simple, modelar primero las clases principales y luego agregar funcionalidades de a poco.

Antes de programar, cada grupo debería responder estas preguntas:

- ¿Qué problema resuelve mi sistema?
- ¿Cuáles son las entidades principales?
- ¿Qué clases necesito?
- ¿Qué atributos tiene cada clase?
- ¿Qué métodos debe tener cada clase?
- ¿Cómo se relacionan las clases entre sí?
- ¿Qué funcionalidades mínimas debe tener el proyecto?

Cierre

Un buen proyecto no es el que tiene más pantallas o más código, sino el que está mejor pensado, mejor organizado y mejor explicado.